

MBA IN SOFTWARE ENGINEERING

Clean Architecture I-II

Prof. Helder Prado Santos

The trustworthiness, originality,
and legality of the educational content presented is
responsibility of the professor.

The total or partial reproduction of this material without
authorization is prohibited.

Law No.
9610/98

The Problem of Coupling

Symptoms of a Bad Architecture

- **Rigidity:** The system is difficult to change because each change affects many other parts.
- **Fragility:** Changes break parts of the system that have no conceptual relation to the altered part.
- **Immobility:** It is difficult to reuse the system in other projects because it involves many risks and effort.

Objective: To transform the "Big Ball of Mud" into a structured system.

Nobody:
My code:



What is Clean Architecture?

Proposed by **Robert C. Martin (Uncle Bob)** in 2012, it is not a "new rule", but a consolidation of best practices (Hexagonal, Onion, Screaming Architecture).



Independence

The system does not depend on libraries or frameworks. They are tools, not the core.



Testability

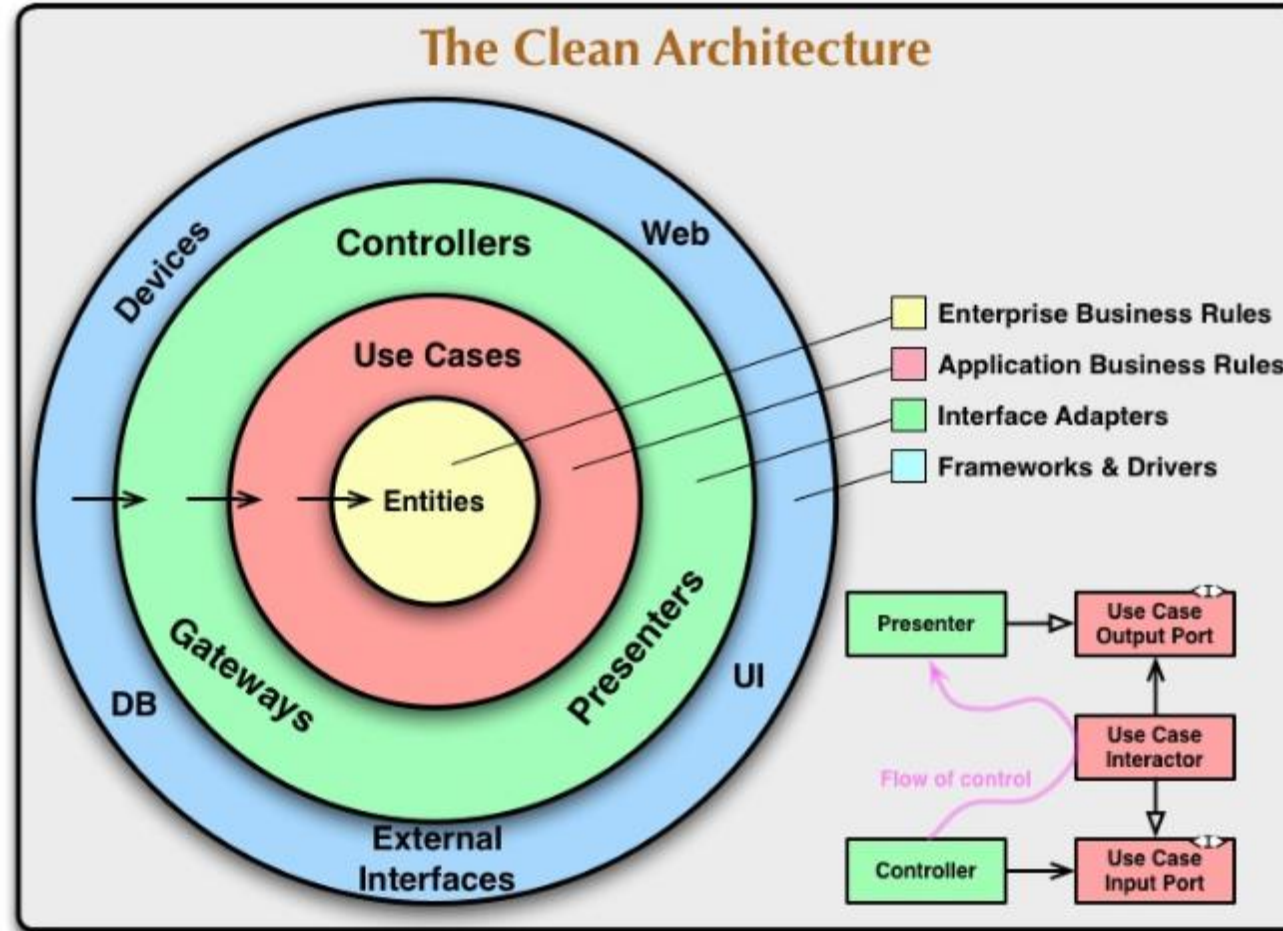
Business rules can be tested without UI, DB, Web Server or any external element.



Database Independence

You can swap Oracle for Mongo, or SQL Server for CouchDB. Business rules don't know what a database is.

The Famous Diagram



The Golden Rule: Dependency

"Source code dependencies must only point inwards."

- Nothing in an inner circle can know anything about an outer circle.
- Names of externally declared classes, functions, or variables should not be mentioned internally.
- The highest level (Rules) does not depend on the lowest level (Details).

Common Violation:

Import an SQL library into your Domain Entity.

1. Entities (Domain)

Corporate Business Rules

These are the application's business objects. They encapsulate the most general and high-level rules.

- If it's an enterprise application, entities can be used by different applications within the company.
- If it's a single application, these are the main domain objects.
- **They do not change** when the page changes, when security changes, or when the database changes.



Entities: Practical Code

```
from uuid import UUID
from domain.task.task_entity import Task
from typing import List

class User:

    id: UUID
    name: str
    tasks: List[Task]

    def __init__(self, id: UUID, name: str):
        self.id = id
        self.name = name
        self.tasks = []
        self.validate()

    def validate(self):
        if not isinstance(self.id, UUID):
            raise Exception("id must be an UUID")

        if not isinstance(self.name, str) or len(self.name) == 0:
            raise Exception("name is required")

    def collect_tasks(self, tasks: List[Task]) -> None:
        self.tasks.extend(tasks)

    def count_pending_tasks(self) -> int:
        count = 0
        for task in self.tasks:
            if task.completed is not True:
                count += 1
```

Notes:

- They have their own identity
- They focus on business rules and behaviors
- They are infrastructure-independent
- They must protect their internal state

2. Use Cases (Application)

Application Business Rules

They contain application-specific rules. They orchestrate the flow of data to and from entities.

- They direct entities to use their corporate business rules.
- They do not affect entities.
- They are not affected by UI, database, or Frameworks.
- Examples: *Create Order, Enroll Student, Generate Statement.*



Use Cases: Dependency Inversion

Before implementing the use case, we define the repository interface. The use case **doesn't know** how to save data to the database; it only knows that it **needs** to save it.

```
import { User } from '../domain/entities/User';

// Contract that the Infrastructure layer must comply with
export interface UserRepositoryInterface {
  save(user: User): Promise<void>;
  findByEmail(email: string): Promise<User | null>;
}
```

This ensures that the Use Case depends on an abstraction, not an implementation (SOLID "D" Principle).

Use Cases: Implementation

```
from domain.__seedwork.use_case_interface import UseCaseInterface
from domain.user.user_repository_interface import
UserRepositoryInterface
from usecases.user.add_user.add_user_dto import (
    AddUserInputDto,
    AddUserOutputDto
)
from domain.user.user_entity import User
import uuid

class AddUserUseCase(UseCaseInterface):

    def __init__(self, user_repository: UserRepositoryInterface):
        self.user_repository = user_repository

    def execute(self, input: AddUserInputDto) -> AddUserOutputDto:

        user = User(
            id=uuid.uuid4(),
            name=input.name
        )

        self.user_repository.add_user(user=user)

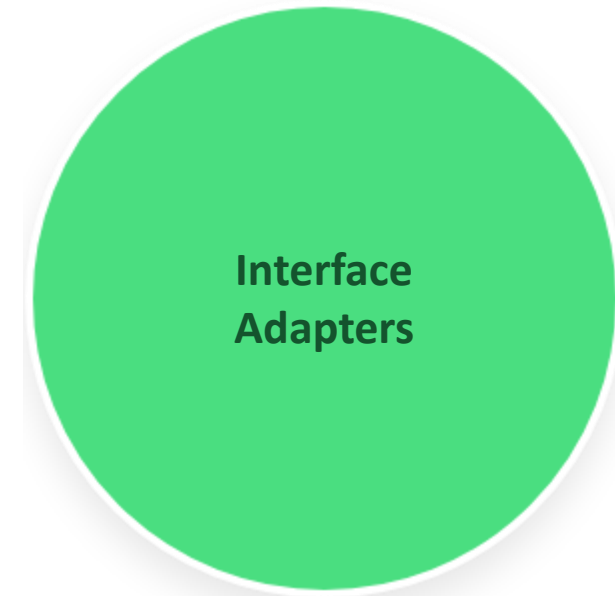
        return AddUserOutputDto(
            id=user.id,
            name=user.name
```

3. Interface Adapters

Data Converters

The goal is to convert data from the most convenient format for use cases and entities to the most convenient format for external agents (Web, database).

- **Controllers:** Receive input (HTTP Request) and convert it to the Use Case's DTO.
- **Presenters:** Receive the Use Case's return and format it for the View (HTML, JSON).
- **Gateways:** Implementations of interfaces for external services.



Adapters: Web Controller

```
export class UserController {
  constructor(private useCase: RegisterUser) {}

  // Receives framework objects (Request, Response)
  async handle(request: Request, response: Response) {
    try {
      const { name, email } = request.body;

      // Calls the use case (Agnostic to HTTP)
      const result = await this.useCase.execute({
        id: UUID.gen(),
        name,
        email,
      });

      return response.status(201).json(result);
    } catch (e) {
      return response.status(400).json({
        error: e.message,
      });
    }
  }
}
```

Responsibilities:

- Decouple HTTP from the Domain.
- Extract data from the request.
- Invoke the Use Case.
- Return the correct answer.

4. Frameworks & Drivers (Infra)

Where the details live

The outermost layer. Generally composed of frameworks and tools such as:

- Database (Postgres, Mongo).
- Web Frameworks (Express, NestJS, Spring).
- Devices (UI, Printers).

We don't write much code here, besides the "glue" code that communicates with the inner circles.



Infra: Repository Implementation

Here we implement the interface defined in the Application layer.

```
import { UserRepositoryInterface } from '../application/repositories';
import { User } from '../domain/entities/User';
import { UserModel } from '../mongoose/UserSchema';

// Concrete implementation using MongoDB (external lib)
export class MongoUserRepository implements UserRepositoryInterface {
  async save(user: User): Promise<void> {
    // Converts Domain Entity → Mongo Model
    await UserModel.create({
      _id: user.id,
      name: user.name,
      email: user.email,
    });
  }

  async findByEmail(email: string): Promise<User | null> {
    const data = await UserModel.findOne({ email });

    if (!data) {
      return null;
    }

    // Reconstructs the Domain Entity
    return new User(
      data._id.toString(),
      data.name,
      data.email
    );
  }
}
```

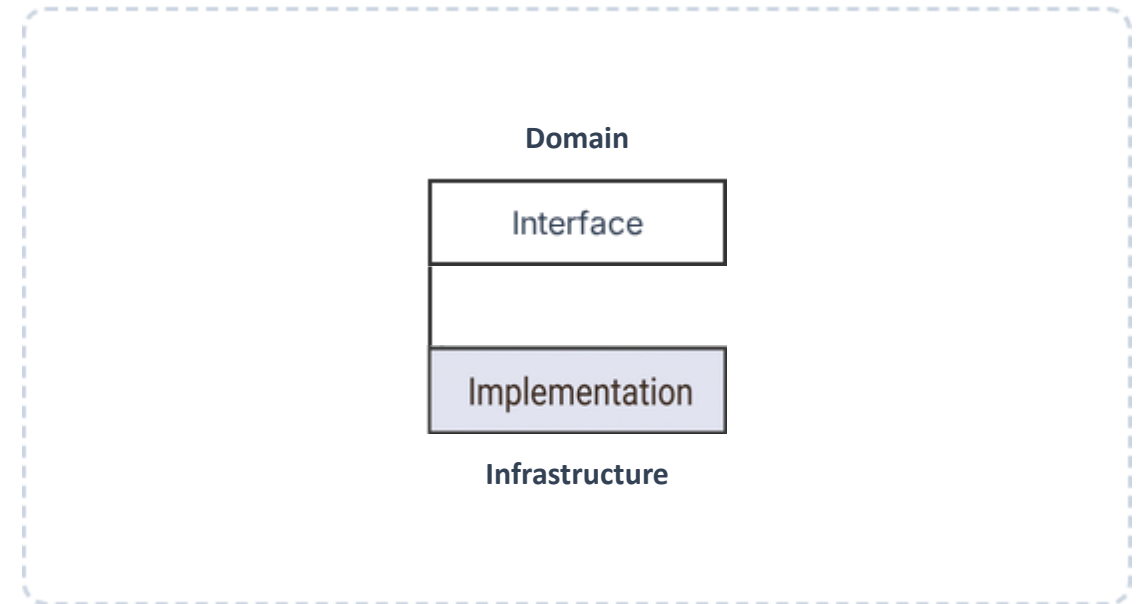
Crossing Boundaries

Control Flow vs. Dependency

The control flow (runtime) can go in any direction (Controller calls UseCase, UseCase calls Repository).

However, the **source code dependency** must always point against the control flow when we cross boundaries outwards.

How? Polymorphism (Interfaces). The Use Case calls a RepositoryInterface interface (which is in the Domain layer), but the MongoRepository implementation is in the Infra layer.



Testing Strategy



Unit Tests

They test Entities and Use Cases. They are fast (ms). They don't use complex Mocks, only pure data.



Integration Tests

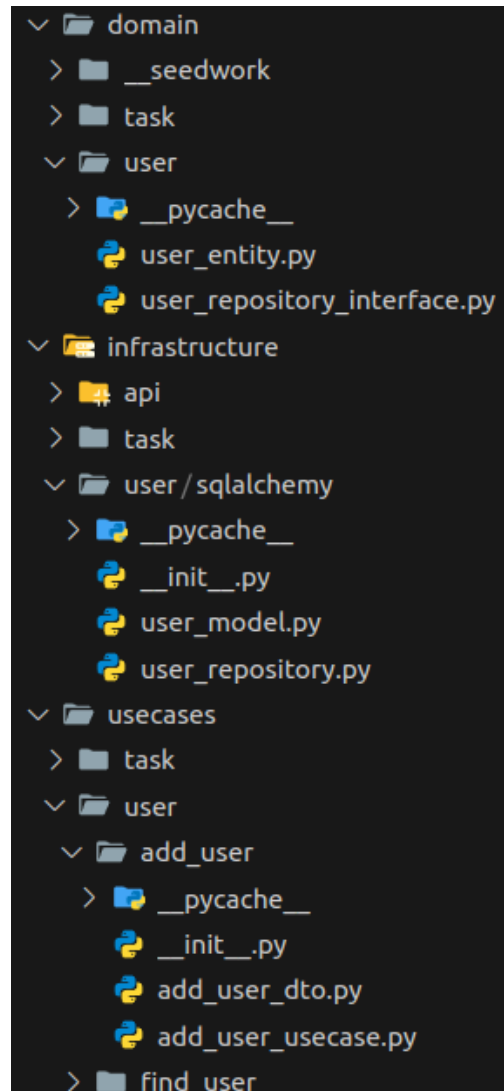
They test the communication between Adapters and Use Cases. They verify if the Controller passes the correct DTO.



E2E Tests

They test the infrastructure. They query the actual database and make real HTTP requests.

Suggested Folder Structure



```

domain
├── __seedwork
├── task
├── user
│   ├── __pycache__
│   ├── user_entity.py
│   └── user_repository_interface.py
├── infrastructure
│   ├── api
│   ├── task
│   └── user / sqlalchemy
│       ├── __pycache__
│       ├── __init__.py
│       ├── user_model.py
│       └── user_repository.py
├── usecases
│   ├── task
│   └── user
│       ├── add_user
│       │   ├── __pycache__
│       │   ├── __init__.py
│       │   ├── add_user_dto.py
│       │   └── add_user_usecase.py
│       └── find_user

```

Screaming Architecture

The folder structure should "scream" about what the system does, not which framework it uses.

When opening the project, you should see "Users", "Orders", "Products", and not just "Models", "Views", "Controllers".

Trade-offs (Not everything is perfect)

Advantages

- Total decoupling.
- Extreme testability.
- Easy long-term maintenance.
- Framework independence.

Disadvantages

- Steep learning curve.
- A lot of "boilerplate" (Many files).
- Mental overhead for simple CRUDs.
- Code duplication (DTO vs Entity).

When to use Clean Architecture?

USE WHEN:

- Medium or large projects, with complex business rules.
- Systems with a long lifespan, which will need to evolve over time.
- Teams with multiple developers, facilitating organization and maintenance.
- Need for automated tests, especially business tests.

AVOID WHEN:

- Small projects or MVPs, where simplicity and speed are a priority.
- Very simple applications (basic CRUD) without complex rules.
- Very short deadlines, where architectural overhead hinders.
- Teams without technical maturity, potentially leading to confusion and overengineering.

Recap

- **Entities:** The core of the domain, containing the most stable business rules.
- **Use Cases:** Orchestrate business rules and define the application flow.
- **Adapters:** Convert data between the external world and the internal domain.
- **Infrastructure:** Technical implementations and replaceable tools.
- **Dependency Rules:** Dependencies always point towards the application's core.

"Good architecture allows critical decisions to be postponed."



Suggested Readings/References

MARTIN, Robert C. Arquitetura limpa: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2018.

MARTIN, Robert C. Código limpo: habilidades práticas do Agile Software. Rio de Janeiro: Alta Books, 2009.

EVANS, Eric. Domain-Driven Design: atacando as complexidades no coração do software. Rio de Janeiro: Alta Books, 2016.

MARTIN, Robert C. (Uncle Bob). The Clean Code Blog. Available at: <https://blog.cleancoder.com>

Thank you!

Helder Prado Santos | [linkedin.com/in/helderprado](https://www.linkedin.com/in/helderprado)

